IUT Nantes
Pôle Sciences et technologie
Nantes Université

GEII - INFO1 - TD/TP-Dessin

Informatique INFO1 – Ressource R1.08 – GEII 1ère Année – IUT Nantes

TP IHM

TP n°1 Édition 2025

Objectifs :

L'objectif de ce TP est de concevoir une interface graphique simple permettant d'afficher un train et ses rails, de les faire suivre la position de la souris, et d'en modifier la couleur via une palette, en utilisant les actions paramétrées fournies par la librairie SDL.

Partie 3:

3.1: Environnement

On définit la constante *LARGEUR_TRAIN*, qui nous sera utile pour tracer nos différentes formes, afin de disposer d'une référence dans nos calculs et de pouvoir l'adapter selon nos besoins.

```
// --- DIMENSIONS ---
#define LARGEUR_TRAIN 40
```

Le fait de définir une largeur fixe permet d'adapter les dimensions d'autres éléments, notamment celles du nez, que nous verrons un peu plus tard.

```
// Avant (Triangle)
for(i = 0; i < LARGEUR_TRAIN; i += 2)
{
    dessineRectanglePlein(x + LONGUEUR_WAGON + (i / 2), y + h + (i / 2), LARGEUR_TRAIN - i, 1, couleur);
}
```

3.2: Les rails

Pour les rails, nous avons découvert une nouvelle mécanique intéressante mettant en avant l'utilisation des pointeurs, notamment pour récupérer la taille de la fenêtre. Cela nous permet d'avoir des rails qui s'adaptent dynamiquement à la taille de celle-ci.

```
// Procedure pour afficher les rails
void afficheRails(int y)
{
    int width, height;
    int i = 0;
    getWindowSize(&width, &height);
```

Puis nous avons tracé deux droites adaptées aux dimensions demandées (42 pixels d'écart).
Sur ces mêmes droites, nous avons ajouté les jonctions des rails grâce à une boucle *for* qui s'adapte à la largeur de la fenêtre.

Affiche rails :

```
// Procédure pour afficher les rails
void afficheRails(int y)
{
   int width, height;
   int i = 0;
   getWindowSize(&width, &height);


   // Lignes horizontales
   dessineLigne(0, y, width, y, IHM_Window_couleur(0, 0, 255));
   dessineLigne(0, y - 42, width, y - 42, IHM_Window_couleur(0, 0, 255));


   // Jonctions des rails
   do
   {
       dessineRectanglePlein(i, y, -42, 3, IHM_Window_couleur(0, 0, 255));
       i = i + 20;
   } while (i < width);
}
```

3.3 : Le train

Pour ce qui est du train, nous avons utilisé des outils similaires à *dessineLigne*, mais cette fois avec des rectangles pleins. Après avoir compris comment adapter le train au mouvement de la souris, nous avons suivi le cahier des charges afin d'obtenir un train composé de :
un premier rectangle, que nous avons nommé ***Base*** ;
un triangle, que nous avons nommé ***Nez*** ;
quatre petits rectangles représentant les **roues**, placés selon les coordonnées définies dans le cahier des charges.

```
// Procédure pour dessiner le train
void afficheTrain(int x, int y, unsigned long couleur)
{
   int h = -LARGEUR_TRAIN; // Hauteur négative pour dessiner vers le haut
   int i;
   unsigned long noir = IHM_Window_couleur(0, 0, 0);


   // Base du Train (Rectangle)
   dessineRectanglePlein(x, y - 1, h + 1, LONGUEUR_WAGON, couleur);

   // Nez (Triangle)
   for(i = 0; i < LARGEUR_TRAIN; i += 2)
   {
       dessineRectanglePlein(x + LONGUEUR_WAGON + (i / 2), y + h + (i / 2), LARGEUR_TRAIN - i, 1, couleur);
   }

    // Roues
   dessineRectanglePlein(x + 20, y + h -5 , 5, 10, noir); // roue haut gauche
   dessineRectanglePlein(x + 20, y - 1, 5, 10, noir); //roue bas gauche
   dessineRectanglePlein(x + 60, y + h -5 , 5, 10, noir); //roue bas gauche
   dessineRectanglePlein(x + 60, y - 1, 5, 10, noir); // roue bas droite
}
```

3.4 : La palette de couleur

Pour la palette, nous avons prélevé les couleurs directement sur les images du TP, car elles n'étaient pas précisées dans le cahier des charges. Pour cela, nous avons fait des captures d'écran et utilisé un outil en ligne capable de fournir le code RVB de l'image.
Ensuite, chaque couleur a été définie dans un tableau nommé *maPalette*.
Nous avons pris la largeur de la fenêtre et l'avons divisée par le nombre de couleurs afin qu'elles soient équilibrées entre elles. Ensuite, chaque palette de couleur demandée dans le cahier des charges fait 10 pixels de haut.
Procédure de la palette :

```
// Procédure pour dessiner la palette
void affichePalette(unsigned long * palette, int nbCouleurs)
{
   int width, height;
   int i;
   getWindowSize(&width, &height);


   for(i = 0; i < nbCouleurs; i += 1)
   {
       dessineRectanglePlein(i * (width / nbCouleurs), height - 10, 10, width / nbCouleurs, palette[i]);
   }
}
```

3.5 : Modification de la couleur

Pour le changement de couleur, nous avons d'abord définie une fenetre de "clic" qui est les 10 pixel de hauteur en bas de page qui correspondent à la palette
Nous avons aussi ajouté une fonction qui était déjà écrite sur le ihm setup qui permettait de détecter lorsque l'utilisateur cliquait ainsi que la position du clic et la couleur associée à cette position. Le programme change ensuite la couleur du train en fonction de la couleur sélectionné

action paramétrée:

```
// Fonction pour déterminer si on clique sur la palette ou non
int getIndiceCouleur(int x, int y, int nbCouleurs)
{
   int width, height;
   getWindowSize(&width, &height);


   if (y < height - 10) return -1; // Le clic est en dehors de la palette


   int largeurCase = width / nbCouleurs;
   if (largeurCase == 0) return 0;


   int indice = x / largeurCase;
   if (indice >= nbCouleurs) indice = nbCouleurs - 1;

   return indice;
}
```

programme dans le main:

```
// Action quand on clique avec la souris
if (boutonClick != 0)
{
    indiceCouleur = getIndiceCouleur(xClick, yClick, NB_COUL);

    if (indiceCouleur >= 0 && indiceCouleur < NB_COUL)
    {
        // CLIC SUR PALETTE : Change la couleur du train
        couleurLocomotive = maPalette[indiceCouleur];
    }
```

3.6 : BONUS

Pour effectuer le bonus nous devions déjà créer les wagons qui suivent le train et imposer une limite de 20 wagons maximum. Pour tracer les wagons nous avons choisis des dimensions adaptées à la “base” du train car aucune n’étaient précisées dans le cahier des charges. Pour ce qui est des roues nous avons repris les mêmes dimensions que celle du train.

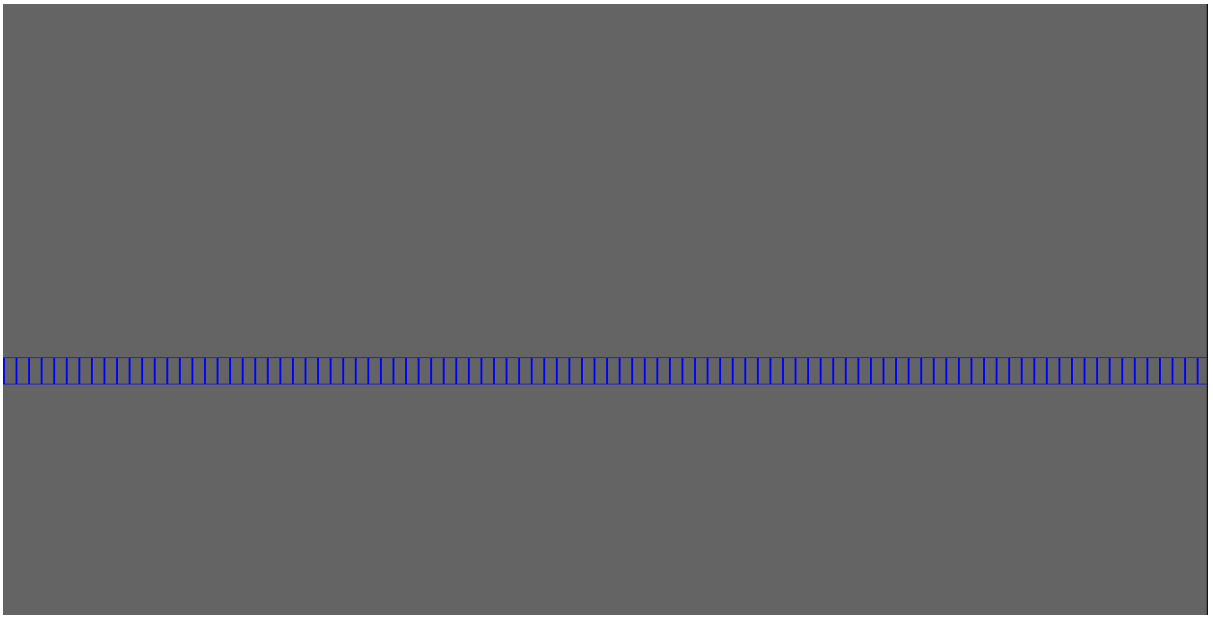
```
// Bonus : Dessin des wagons
void afficheWagon(int x, int y, unsigned long couleur)
{
    int h = -LARGEUR_TRAIN;
    unsigned long noir = IHM_Window_couleur(0, 0, 0);

     //Wagon = corps du train
    dessineRectanglePlein(x, y - 1, h + 1, LONGUEUR_WAGON, couleur);

     // Roues
    dessineRectanglePlein(x + 20, y + h - 5, 5, 10, noir);
    dessineRectanglePlein(x + 20, y - 1, 5, 10, noir);
    dessineRectanglePlein(x + 60, y + h - 5, 5, 10, noir);
    dessineRectanglePlein(x + 60, y - 1, 5, 10, noir);
}
```

Nous avons ensuite dû faire en sorte que lorsque l’utilisateur clique hors de la palette, un wagon s’affiche à la suite du train avec un espacement de 10 pixels. Pour celà nous avons juste fait if dans le main qui permettrait que lorsque l’utilisateur ne clique pas sur la palette on appelle la procédure pour créer un wagon. Pour ce qui est de la couleur des wagons, peu de précisions étaient apportées dans le cahier des charges, nous avons pris des libertés. Avec la fonction rand nous avons fait en sorte qu’un nombre aléatoire soit choisi en index du tableau de la palette pour que la couleur de chaque wagon soit aléatoire parmi les couleurs du tableau.

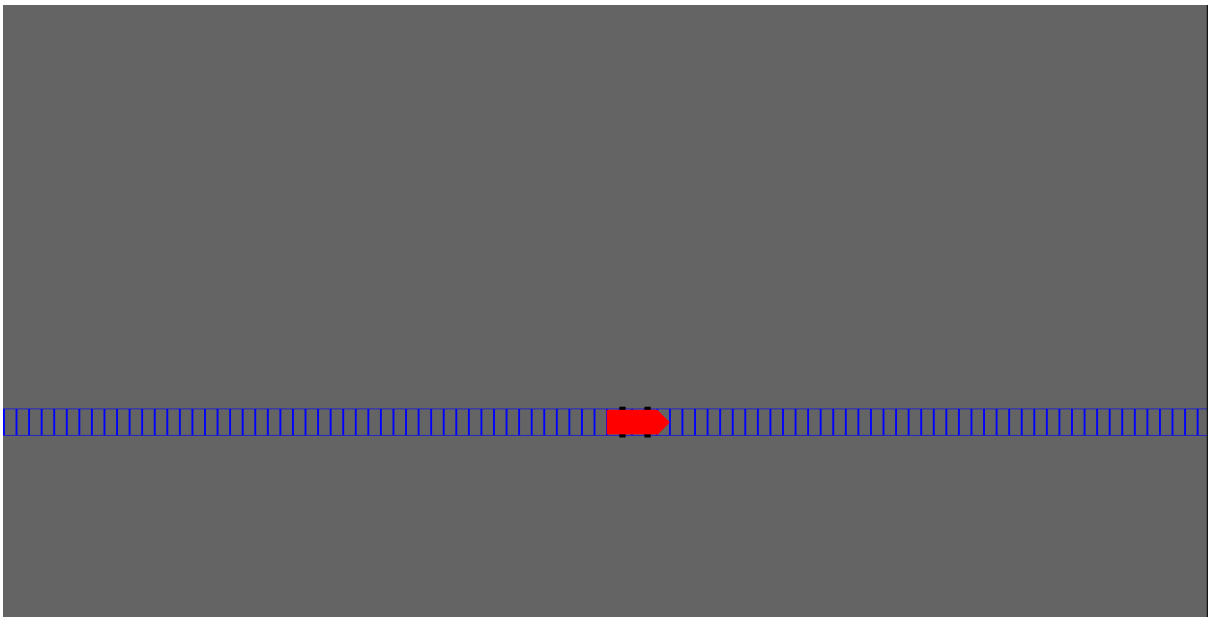
```
// Action quand on clique avec la souris
if (boutonClick != 0)
{
    indiceCouleur = getIndiceCouleur(xClick, yClick, NB_COUL);

    if (indiceCouleur >= 0 && indiceCouleur < NB_COUL)
    {
        // CLIC SUR PALETTE : Change la couleur du train
        couleurLocomotive = maPalette[indiceCouleur];
    }
    else
    {
        // CLIC HORS PALETTE : Ajoute un wagon avec une couleur aléatoire
        if (nbWagons < MAX_WAGONS)
        {
            // Tirage d'un nombre aléatoire entre 0 et 4 (NB_COUL - 1)
            int randomIdx = rand() % NB_COUL;

            // On assigne cette couleur aléatoire au nouveau wagon
            couleursWagons[nbWagons] = maPalette[randomIdx];

            nbWagons++;
        }
    }
}
```

Exécution du programme étapes par étapes:

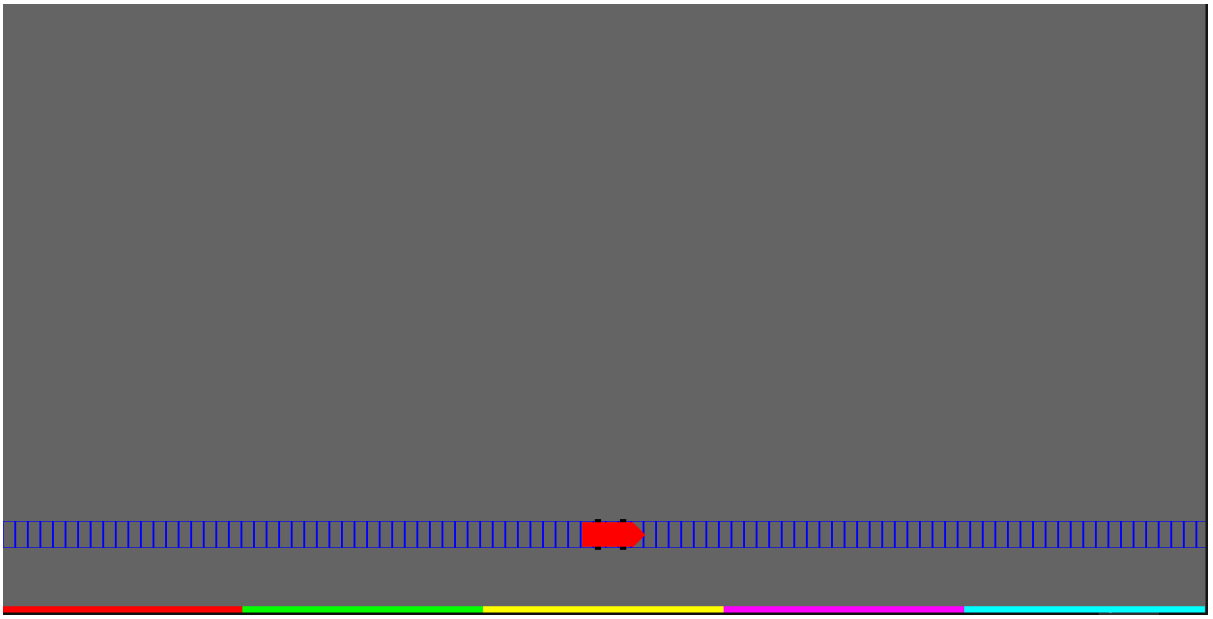
Afficher les rails:



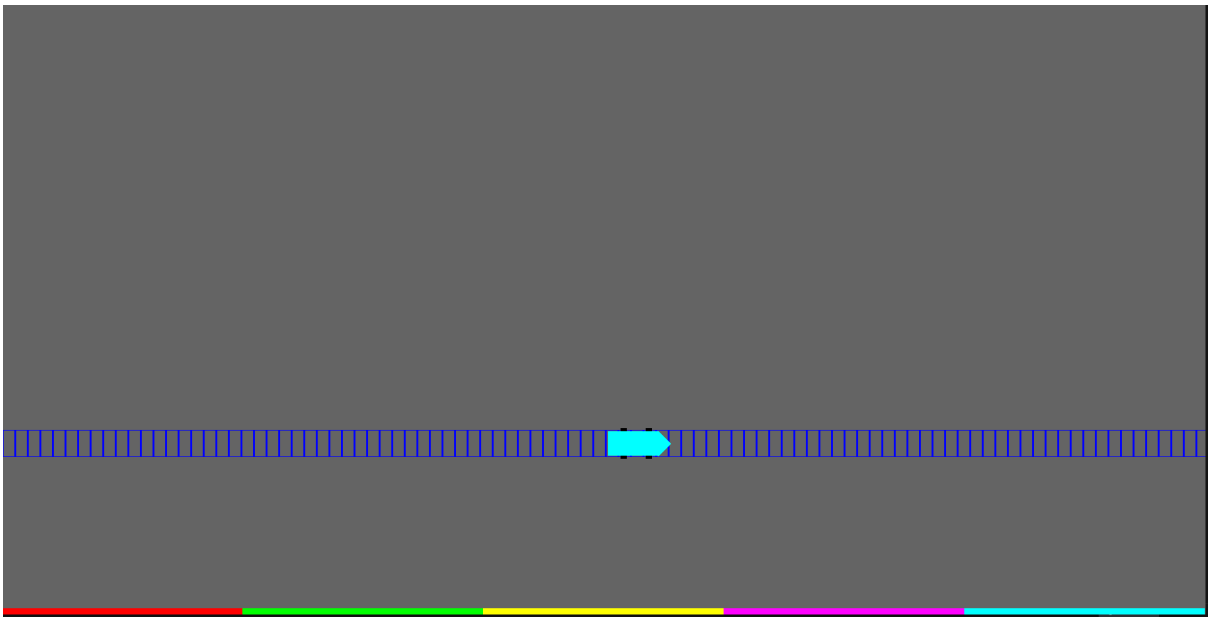
Afficher le train:

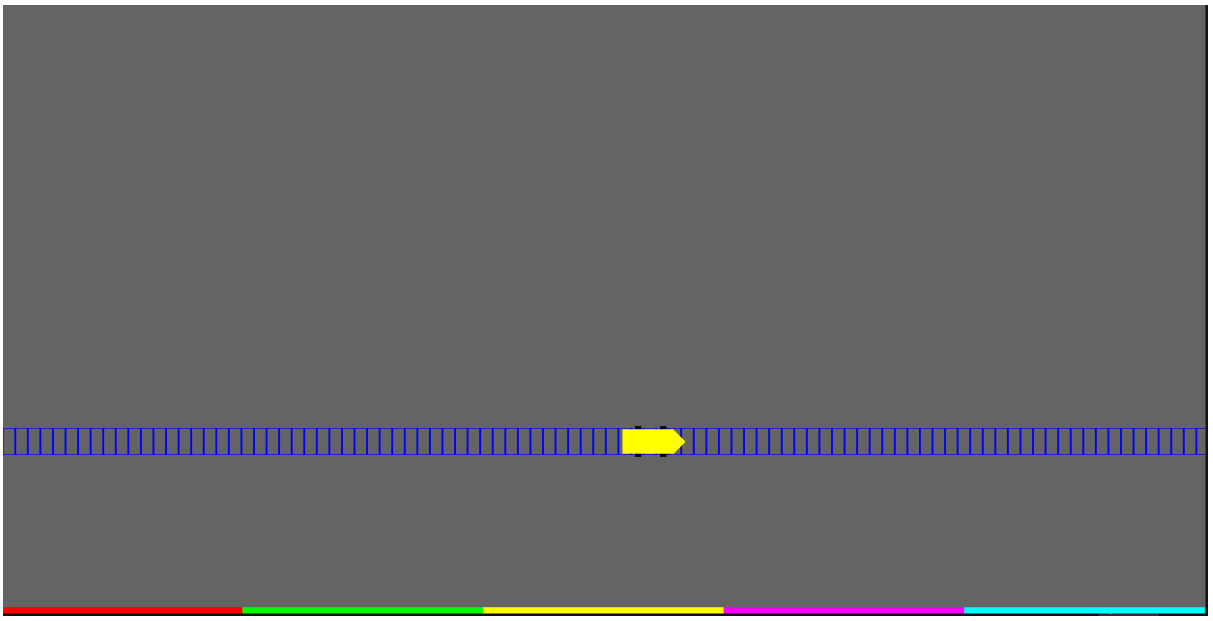
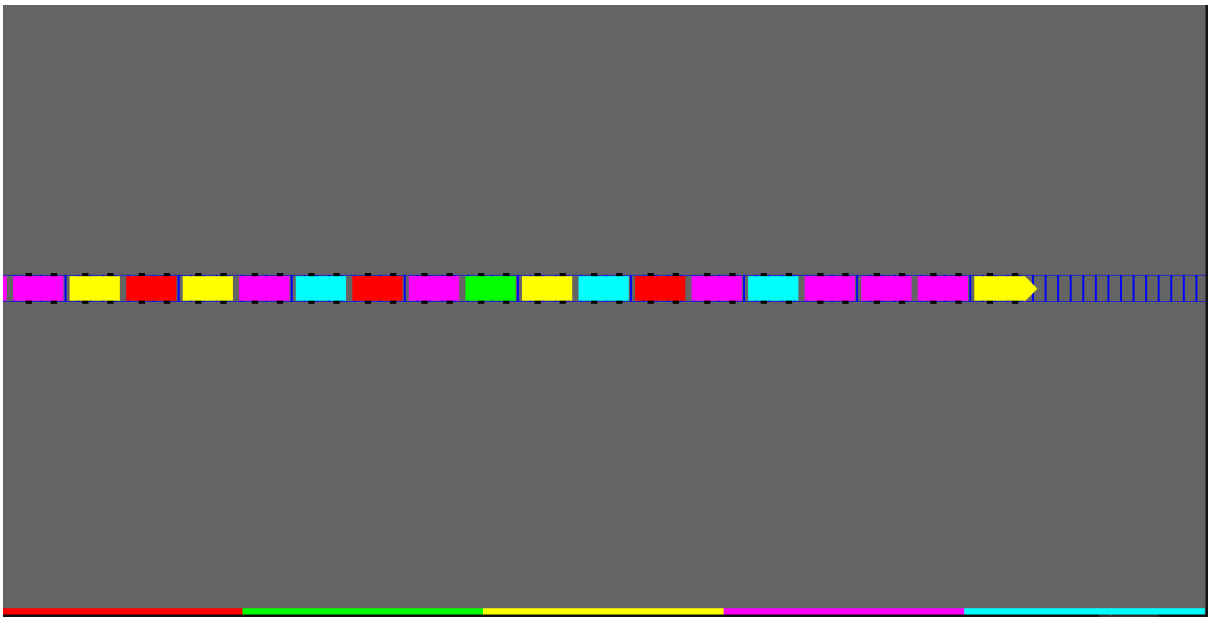


Afficher la palette de couleur:



Modification de la couleur:





Bonus: (couleur aléatoire)